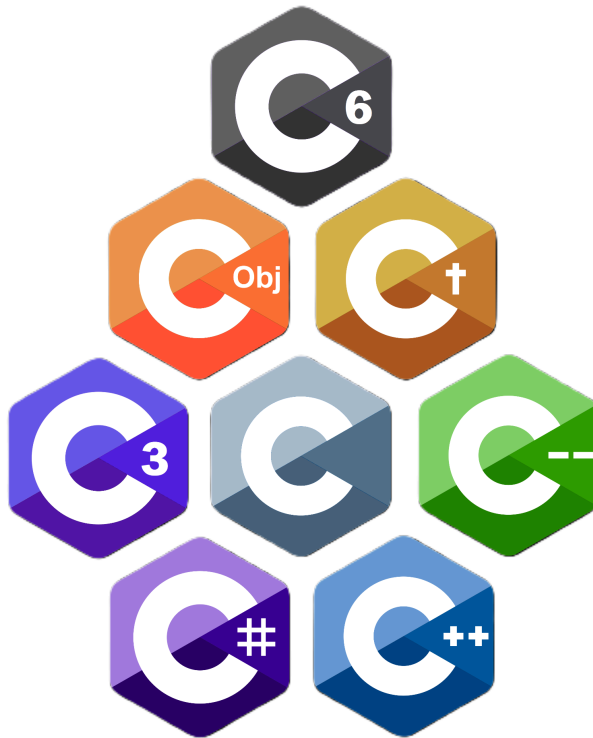


Materia: Laboratorio de Programación
EEST N.º 1 - "Eduardo Ader" Vicente López
5º año 3º Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

1º CUATRIMESTRE

ACTIVIDAD 3: ENFOQUE MODULAR Y DISEÑO TOP-DOWN - LPR 5º 3º A-B



Estudiante: Mikaela Batirola
Curso: 5º 3º
Profesores: Yamil Ganduglia y York Muñoz Mansilla

Repositorio Git Hub:
https://github.com/mikabatirola434-svg/CALCULO_AREA

1. ¿Por qué el compilador no sabría qué hacer cuando intente leer main() si borráramos el prototipo?

El compilador no sabría que existe la función `calcularAreaCirculo()` cuando la función es utilizada dentro de `main()`. El prototipo sirve para avisarle al compilador el nombre de la función, el tipo de dato que devuelve y los parámetros que necesita.

2. ¿Qué problema físico o geométrico ocurriría si intentáramos calcular el área de un círculo con un radio de 0 cm o de -5 cm? ¿Por qué es importante controlar esto?

Un radio de 0 cm no representa un círculo con superficie, por lo que su área sería 0. Un radio de -5 cm no es válido porque un radio no puede ser negativo. Por eso es importante controlar el dato ingresado y evitar que el programa trabaje con valores geométricamente incorrectos.

3. ¿Por qué es más seguro declarar valores como `const` en lugar de usar variables comunes?

Porque `const` hace que el valor no pueda modificarse durante la ejecución del programa. En este caso, el valor de `PI` debe mantenerse fijo. Esto ayuda a evitar modificaciones accidentales y hace que el programa sea más seguro y confiable, especialmente en sistemas donde un cambio incorrecto podría provocar errores.

Figura 1

```
src > C:\main.cpp > main()
23 int main() {
60     cout << "\n[SISTEMA] Dibujando escala de la Diana en la RAM..." << endl;
61
62     if (radioEstudiante <= 5.0) {
63
64         cout << "      .-.-.-.      " << endl;
65         cout << "    /  X  \  " << endl;
66         cout << "   \  *  /   " << endl;
67         cout << "    '---'    " << endl;
68
69     } else if (radioEstudiante <= 12.0) {
70
71         cout << "      .-.-.-.      " << endl;
72         cout << "    /  .-.-. \  " << endl;
73         cout << "   | /  O  \ |   " << endl;
74         cout << "   | \  *  / |   " << endl;
75         cout << "   \  '---' /    " << endl;
76         cout << "    '---'    " << endl;
77
78     } else {
79
80         cout << "      .-.-.-.      " << endl;
81         cout << "    /  .-.-. \  " << endl;
82         cout << "   | /  .-.- \ |   " << endl;
83         cout << "   | \  X  / |   " << endl;
84         cout << "   \  '---' /    " << endl;
85         cout << "    '---'    " << endl;
86     }
87 }
```

```
PS C:\Users\Alumnos\Desktop\Calculoarea> g++ src/main.cpp -o src/area.exe
PS C:\Users\Alumnos\Desktop\Calculoarea> .\src\area.exe

CALCULADORA DE AREA MODULAR - ESTUDIANTE: Mikaela Batirola

-> Ingrese el radio del circulo/diana (en cm): 150
El area del circulo es: 70685.8 cm2

[SISTEMA] Dibujando escala de la Diana en la RAM...
      .-.-.-. \
    /  .-.-. \ |
   | /  X  / | -> [DIANA GIGANTE DE COBERTURA]
   | \  *  / |
   \  '---' /
    '---'

```

Ejecución del programa “Calculadora de Área Modular”

Nota explicativa: En la imagen se observa la ejecución del programa en la terminal de Visual Studio Code. Se ingresó un radio de **150 cm** y el programa calculó correctamente el área del círculo, obteniendo **70,685 cm²**. Luego, mostró en la consola la representación correspondiente a una **diana estándar de tiro**

Figura 2

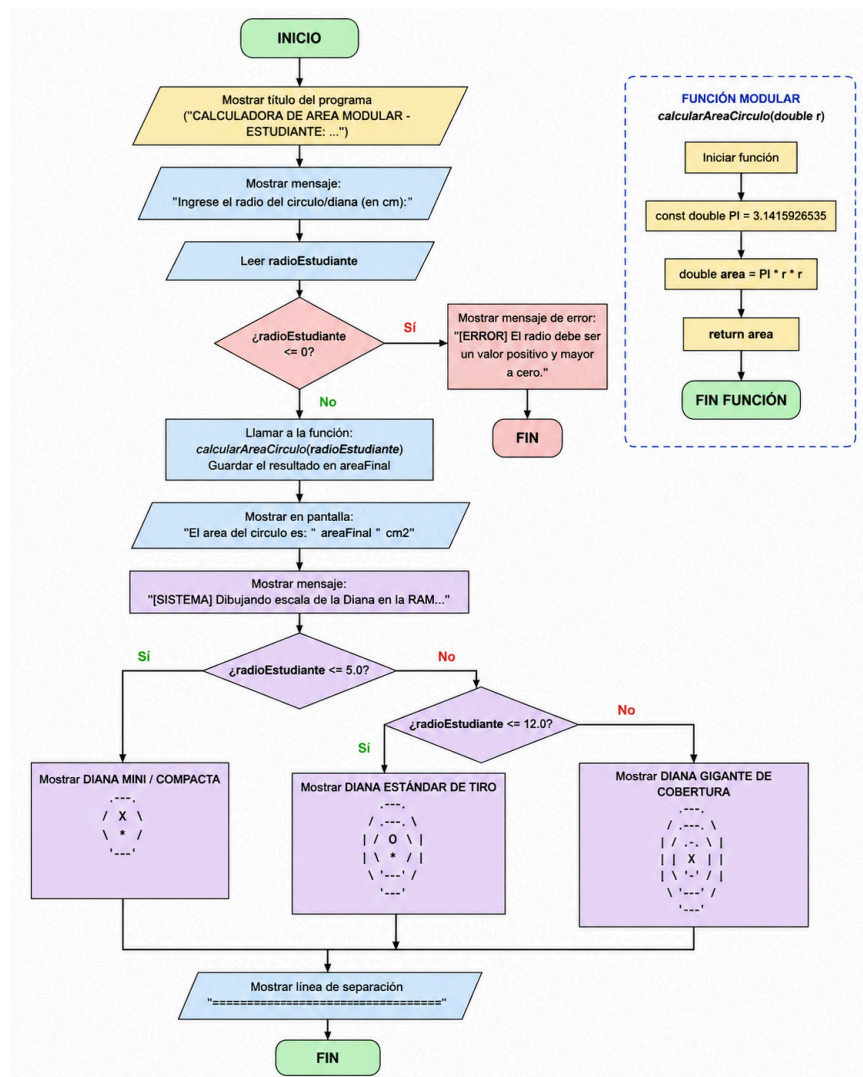


Diagrama de flujo del programa “Calculadora de Área Modular”

Nota explicativa:

El diagrama de flujo representa el funcionamiento del programa, desde el ingreso y validación del radio hasta el cálculo del área mediante la función modular. Luego, muestra el resultado y selecciona la representación de la diana según el valor del radio ingresado.